

PROJECT EXECUTABLE FILES

Submission draft prepared from the supplied Learnmate code screenshots and the provided template

Field	Details
Project name	Learnmate
Project type	Web application — Node.js / Express backend
Database	MongoDB (Mongoose)
Team ID	To be filled by the team
Submission date	To be confirmed
Maximum marks	3 marks (as stated in the supplied template)

Purpose

This document summarizes the executable project files, folder structure, setup information, and known limitations for evaluator review. It is based only on the screenshots and the supplied “Project Executable Files” template. Items not visible in the screenshots are marked **TO BE CONFIRMED** rather than assumed.

Important: This is a submission-document draft, not the complete runnable source-code package. The screenshots show selected files only; the full source folder, package.json, and actual environment values must be included separately in the final submission.

Step 1 — Submission checklist

S.No	Item to submit	Status / note
1	Complete source code (all files and folders)	Partial evidence provided as screenshots. Submit the actual project folder / repository.
2	README / setup guide	This document includes draft run instructions; verify and add a README.md to the project.
3	Dependency file	package.json and package-lock.json are shown in the Explorer. Include both.
4	Database schema / seed files	Mongoose models shown: User.js and Material.js. Seed data: TO BE CONFIRMED.
5	Environment configuration	.env is present locally. Submit .env.example only; never submit real secrets.
6	Deployed application URL	TO BE CONFIRMED / not visible in the supplied material.
7	APK / executable binary	Not applicable based on the visible Node.js web-backend structure, unless a screenshot is provided.
8	Dockerfile / containerization	TO BE CONFIRMED; not visible in the screenshots.
9	Test files and test results	TO BE CONFIRMED; no test files/results are visible.
10	Demo video / walkthrough	TO BE CONFIRMED; provide a link if required.

Step 2 — File / folder structure

The following structure is transcribed from the supplied VS Code Explorer screenshots. Only visible files are listed.

```
Learnmate/
├── .postman/           [folder shown]
├── .vscode/           [folder shown]
├── Code_Files/
│   ├── node_modules/   [dependency folder; do not submit if platform says to exclude]
│   └── src/
│       ├── controllers/
│       │   ├── adminController.js
│       │   ├── authController.js
│       │   └── materialController.js
│       ├── middleware/
│       │   ├── auth.js
│       │   └── upload.js
│       ├── models/
│       │   ├── Material.js
│       │   └── User.js
│       ├── routes/
│       │   ├── admin.js
│       │   ├── auth.js
│       │   └── materials.js
│       ├── utils/
│       │   ├── db.js
│       │   ├── gemini.js
│       │   └── tokens.js
│       └── uploads/
├── .env                [local secret file – do not upload]
├── index.js
├── package-lock.json
├── package.json
├── README.md
├── node_modules/
└── postman/           [folder shown]
```

Main components identified from the screenshots

Component	Observed responsibility
index.js	Loads environment variables, configures Express and CORS, mounts /api/auth, /api/materials
src/utils/db.js	Connects to MongoDB using MONGO_URI.
src/middleware/auth.js	Verifies a Bearer JWT and provides an admin-only role check.
src/middleware/upload.js	Uses Multer disk storage; accepts .txt, .md and .pdf files; configured maximum file size is 5 M
src/models/User.js	Mongoose user model with name, email, password and role; hashes passwords using bcrypt
src/models/Material.js	Mongoose material model with user, title, content, filename, summary, flashcards, quiz and s
src/utils/gemini.js	Initializes Google Generative AI and selects the gemini-3.7-flash model; includes retry handl

Step 3 — Deployment / access details

Field	Details
Hosted / deployed URL	TO BE CONFIRMED
Demo login credentials	Create a dedicated evaluator account. Do not include a real personal password here.
Platform / hosting provider	TO BE CONFIRMED
Repository link	TO BE CONFIRMED
Demo video link	TO BE CONFIRMED
Database hosting	MongoDB connection is configured through MONGO_URI; hosting/provider is TO BE

Step 4 — Run instructions (draft)

These steps reflect the visible Node.js / Express entry point and environment variables. Confirm the exact npm scripts and required variable names against package.json and the full source before submission.

Step	Instruction
1	Install a supported Node.js version and npm on the computer.
2	Extract / open the Learnmate project folder in a terminal.
3	Install dependencies using <code>npm install</code> from the folder containing package.json.
4	Create a local .env file. Configure at least the variables referenced in the screenshots: MONGO_URI, JWT_ACCO
5	Start MongoDB or provide a reachable MongoDB connection string in MONGO_URI.
6	Start the backend using the start command defined in package.json. If no script is defined, the visible entry file is
7	Check the terminal for the server and database connection messages.
8	Open the health-check endpoint at <code>http://localhost:5000/</code> (or the configured PORT). The visible code retu
9	Use the frontend/client and Postman collection, if included, to test authentication, materials and admin routes. TH

Environment safety

Do not upload the actual .env file or publish API keys, JWT secrets, database credentials, or real user passwords. Provide a sanitized .env.example with placeholder values instead.

Step 5 — Known issues / limitations

S.No	Known issue / limitation	Workaround / status
1	The supplied material contains screenshots of selected the complete project folder repository, before final submission.	Attach the complete project folder repository, before final submission.
2	Deployment URL, hosting provider, repository URL these fields will be deployed now recording; otherwise mark N/A visible.	URL these fields will be deployed now recording; otherwise mark N/A visible.
3	Exact package scripts and complete dependencies are visible in package.json and package-lock.json; test installation on a clean environment.	Verify package.json and package-lock.json; test installation on a clean environment.
4	The exact Gemini API environment-variable name should be checked in the service.js and document it in .env.example.	Consistent should be checked in the service.js and document it in .env.example.
5	No automated test files or test results are visible.	Add basic API tests and attach a test-results summary, or state N/A.
6	Upload middleware accepts only .txt, .md and .pdf files the demo uses as uploaded files.	Upload middleware accepts only .txt, .md and .pdf files the demo uses as uploaded files.

Final verification before submission

Check	Done
Complete source folder included (not screenshots alone)	■
package.json and package-lock.json included	■
.env.example included; real .env excluded	■
README run steps tested on a clean environment	■
Demo account created and verified	■
Deployment / repository / video links checked	■
Known limitations updated to match the final build	■

Prepared from the user-provided project screenshots and the uploaded "Project Executable Files" template. Verify all TO BE CONFIRMED fields and run the application before submitting.